

Docker Einstieg

CLI-Befehle ausführen



Beschreibung

Dieses Dokument bietet den Einstieg in Docker-Images und Containers.

Inhaltsverzeichnis

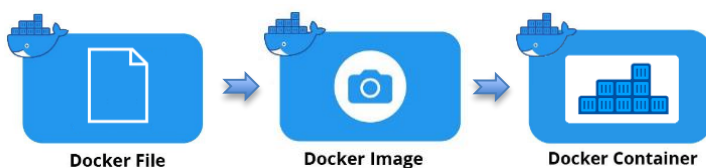
1	DOCKER QUICKSTART	2
1.1	EINLEITUNG	2
1.2	WAS SIND DOCKER-IMAGES?	2
1.3	WOHER KOMMEN DOCKER-IMAGES?	2
1.4	WAS SIND DOCKER-CONTAINER?	3
1.5	WAS IST DOCKER-FILE?	3
1.6	DOCKER BEFEHLE	3
2	ÜBUNGEN MIT DOCKER CLI-BEFEHLEN	4
2.1	EINLEITUNG	4
2.2	IMAGES VERWALTEN	4
2.2.1	<i>Welche Images sind auf meinem Computer gespeichert?</i>	<i>4</i>
2.2.2	<i>Wie kann ich Images entfernen?</i>	<i>4</i>
2.2.3	<i>Aufgabe 1 - Reinen Tisch machen</i>	<i>5</i>
2.3	DOCKER UND DOCKER CLI-BEFEHLE VERTIEFEN	6
2.3.1	<i>Aufgabe 2 – Ein erstes Image laden</i>	<i>6</i>
2.3.2	<i>Aufgabe 3 – Nginx verwenden</i>	<i>7</i>
2.3.3	<i>Aufgabe 4 – Mit Docker-Compose Dateien umgehen</i>	<i>7</i>
3	WEITERFÜHRENDE AUFGABEN	9
3.1	AUFGABE 5 – NGINX PARAMETRISIEREN UND STARTEN	9
4	LERNKONTROLLE	10
4.1	REPETITION 1 – DOCKER CLI-BEFEHLE	10
4.2	REPETITION 2 – CLI-BEFEHLE GRUPPIEREN	11
4.3	REPETITION 3 - REPETITIONSFRAGEN DOCKER-CLI-BEFEHLEN	11

1 Docker Quickstart

1.1 Einleitung

Docker-Images sind statische, Read-Only-Vorlagen, die verwendet werden, um Docker-Container zu erstellen. Ein Docker-Image enthält alle notwendigen Dateien, Befehle und Informationen, um eine Anwendung in einem Container auszuführen. Ein Docker-Container ist eine lauffähige Instanz eines Docker-Images, die auf einem Host ausgeführt wird. Ein Image kann verwendet werden, um mehrere Container zu erstellen, die alle identisch sind und die gleiche Anwendung ausführen.

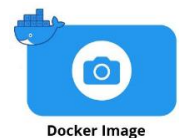
Sie können sich dies vorstellen wie bspw. in einer Objektorientierten Programmiersprache die Klassen und die Objekte. Während die Docker-Images (Klassen) die Vorlagen darstellen, sind die Docker-Container (Objekte) die instanziierte Laufzeitinstanzen, die mit spezifischen Eigenschaften laufen.



1.2 Was sind Docker-Images?

Docker-Images sind ein wichtiger Bestandteil der Entwicklung von Anwendungen in Containern. Sie ermöglichen es Anwendungen in einer bekannten und kontrollierten Umgebung auszuführen, die auf jedem Host gleich ist auf dem die Container-Runtime ausgeführt wird.

Ein Docker-Image enthält alle notwendigen Ressourcen für eine Anwendung, wie das Betriebssystem, die Anwendung selbst sowie alle Abhängigkeiten. Dadurch kann sichergestellt werden, dass die Anwendung auf jedem Host dieselben Ergebnisse liefert.



Die Entwickler:innen können sicherstellen, dass die Umgebung, in der die Anwendung auf ihren Entwicklungsmaschinen ausgeführt wird, der Umgebung entspricht, in der die Anwendung auch in der Produktion ausgeführt werden wird. Dies reduziert die Fehler durch unterschiedliche Laufzeitumgebungen.

1.3 Woher kommen Docker-Images?

Eine Docker-Registry ist ein Ort, an dem Docker-Images gespeichert und verwaltet werden können. Eine der bekanntesten und am häufigsten verwendeten Registry ist Docker-Hub. Es handelt sich hierbei um eine öffentliche Registry, auf der Sie Ihre Images hochladen und anderen zur Verfügung stellen können.

Docker Hub ermöglicht Ihnen, Ihre Images zu speichern, zu verwalten und zu teilen. Es ermöglicht es Ihnen auch, Versionen der Images zu verwalten, was besonders nützlich ist, wenn Sie sich beispielsweise in der Entwicklungs- oder einer Produktionsumgebung befinden.

<https://hub.docker.com>

Es gibt auch private Registry, die es Unternehmen ermöglichen, ihre Images intern zu speichern und zu verwalten, anstatt sie öffentlich zugänglich zu machen. Dies ermöglicht es Unternehmen, ihre Images sicher zu speichern und sicherzustellen, dass nur autorisierte Personen auf sie zugreifen können.

1.4 Was sind Docker-Container?

Docker-Container sind eine Art von virtuellen Maschinen, die es ermöglichen, Anwendungen und deren Abhängigkeiten in isolierten Umgebungen auszuführen. Im Gegensatz zu virtuellen Maschinen, die ihre eigene Kopie des Betriebssystems und der Hardware auf einem physischen Server ausführen, teilen sich Docker-Container das Betriebssystem des Hosts. Dies bedeutet, dass Container weniger Ressourcen benötigen und schneller gestartet werden können als bspw. virtuelle Maschinen.

Ein Docker-Container ist eine Instanz eines Images mit den Konfigurationsoptionen, die beim Starten des Containers angegeben werden. Ein Image enthält die Anwendung und ihre Abhängigkeiten, während die Konfigurationsoptionen die Umgebungsvariablen, Netzwerk- und Speicheroptionen usw. steuern.

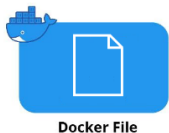


Einmal gestartet, läuft ein Container in seiner eigenen isolierten Umgebung und hat Zugriff auf die Ressourcen, die ihm zugewiesen wurden. Er kann mit anderen Containern auf demselben Host kommunizieren und auf das Netzwerk und die Speicher des Hosts zugreifen, wenn der Container entsprechend konfiguriert ist.

→ Docker-Container ermöglichen Anwendungen in einer definierten Umgebung auszuführen, die der Umgebung entspricht, in der sie entwickelt wurden.

1.5 Was ist Docker-File?

Ein Docker-File ist eine einfache Textdatei mit einer Reihe von Befehlen oder Anweisungen. Diese Befehle und Anweisungen beschreiben das Image und werden nacheinander ausgeführt, um Aktionen für dem Basis-Image auszuführen und so ein neues Docker-Image zu erstellen.



1.6 Docker Befehle

Für die Entwicklung von Anwendungen ist die Handhabung von Docker-Images auf dem Entwicklungscomputer enorm wichtig. Diese werden wir in den folgenden Kapiteln genauer anschauen.

2 Übungen mit Docker CLI-Befehlen

2.1 Einleitung

Folgen Sie den Übungen und machen Sie sich allenfalls Notizen. Die Befehle, welche Sie in diesem Kapitel lernen müssen Sie im Verlauf des Modules und der ganzen Lehrzeit auswendig kennen.

2.2 Images verwalten

2.2.1 Welche Images sind auf meinem Computer gespeichert?

Dieser Befehl zeigt die Images, die auf Ihrem Computer zurzeit gespeichert sind.

```
$ docker images
```



```
REPOSITORY TAG IMAGE ID CREATED SIZE
```

Dieser Output enthält keine Images. Offenbar wurden auf diesem Computer keine Images heruntergeladen.

Hier ein weiteres Beispiel mit vorhandenen Images:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
bitnami/zookeeper	latest	414728774df0	6 weeks ago	510MB
bitnami/kafka	latest	41893ad102d9	6 weeks ago	551MB
mariadb	10.6	d5ef13d90d77	7 weeks ago	389MB
mariadb	10.6.11	d5ef13d90d77	7 weeks ago	389MB
mysql	latest	7b6f3978ca29	7 weeks ago	550MB
mariadb	10.8.6	240ba099d96b	8 weeks ago	361MB
mysql	5.7	2d44289af685	8 weeks ago	495MB
wordpress	latest	3cd489f44849	2 months ago	564MB
mariadb	10.8.4	f1fcd550a240	4 months ago	302MB
mariadb	10.9.2	46ecf4f48e29	4 months ago	302MB
phpmyadmin/phpmyadmin	latest	4a4023c7e22a	8 months ago	510MB
mariadb	10.5.15	eb7ed9035465	9 months ago	395MB

Dieser Output zeigt eine ziemlich lange Liste mit Images.

→ Von Zeit zu Zeit lohnt es sich diese Liste zu kontrollieren und Images, die nicht mehr gebraucht werden zu entfernen. Images enthalten keine persönlichen Konfigurationen und können demnach immer wieder neu von der entsprechenden Registry geladen werden.

2.2.2 Wie kann ich Images entfernen?

Mit dem folgenden Befehl `docker image` können Sie ein Image entfernen. Z.B. `rm eb7ed9035465`, `rm` heisst «remove». Danach wird die ID des Images angegeben.

```
$ docker image rm eb7ed9035465
```

Sie können auch mehrere Images gleichzeitig löschen. Grundsätzlich ist es nicht erforderlich, die gesamte ID anzugeben, es reicht aus, wenn Sie genügend Zeichen der ID angeben, um sie eindeutig zu identifizieren.

```
$ docker image rm eb 4a 46
```

Hier ein Sonderfall:

Manchmal kann ein Image nicht gelöscht werden, wenn ein Docker-Container aktiv oder bereit ist. In einem solchen Fall müssen Sie den Container entfernen bzw. zuerst stoppen und dann entfernen, bevor Sie ein nicht mehr benötigtes Docker-Image entfernen können.

Überprüfen Sie, ob ein Docker Container aktiv oder bereit ist, welches ein bestimmtes Image benötigt:

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e6f6b577e533	phpmyadmin/phpmyadmin	"/docker-entrypoint.s..."	8 days ago	Exited (255) 18 minutes ago	0.0.0.0:8090->80/tcp	firstdatabase-phpmyadmin-1
4985cd2f495e	mariadb:10.8.6	"docker-entrypoint.s..."	8 days ago	Up 18 minutes	0.0.0.0:3306->3306/tcp	firstdatabase-mariadb-1
66295a923412	phpmyadmin/phpmyadmin	"/docker-entrypoint.s..."	9 days ago	Exited (255) 9 days ago	0.0.0.0:8090->80/tcp	transactions-phpmyadmin-1
fe5fb9b9d504	mysql	"docker-entrypoint.s..."	9 days ago	Exited (137) 9 days ago		transactions-mysqldb-1
cfb3034b1894	phpmyadmin/phpmyadmin	"/docker-entrypoint.s..."	6 weeks ago	Exited (0) 5 weeks ago		database-phpmyadmin-1
76c38bc59b75	mariadb:10.8.6	"docker-entrypoint.s..."	6 weeks ago	Exited (137) 8 days ago		database-mysqldb-1
d1df02b2b276	phpmyadmin/phpmyadmin	"/docker-entrypoint.s..."	6 weeks ago	Exited (0) 6 weeks ago		m346-ref-card-03-phpmyadmin-1
7ad4db567204	mariadb:10.9.2	"docker-entrypoint.s..."	6 weeks ago	Exited (137) 6 weeks ago		m346-ref-card-03-database-1

In diesem Beispiel sieht man, dass der Container 4985cd2f495e auf dem Image mariadb:10.8.6 basiert. Das entsprechende Image kann also nicht ohne Weiteres gelöscht werden.

Da der Container aktiv ist, muss er zuerst gestoppt werden:

```
$ docker stop 4985cd2f495e
```

Danach kann er entfernt werden. Machen Sie das mit allen Containern, welches auf dem entsprechenden Image basieren.

```
$ docker rm 4985cd2f495e
```

Nun können Sie das Image über den entsprechenden Befehl und seiner ID entfernen.

```
$ docker image rm 240ba099d96b
```

→ Von Zeit zu Zeit lohnt es sich diese Liste zu kontrollieren und Container, die nicht mehr gebraucht werden zu entfernen. Container enthalten keine Daten und können demnach immer wieder neu vom entsprechenden Image gestartet werden.

Noch ein Sonderfall:

Ein Docker-Image lässt sich nicht löschen, da ein Volume existiert. Mit dem folgenden Befehlen lassen sich die Volumes auflisten und löschen.

```
$ docker volume ls
```

```
$ docker volume prune
```

→ **Achtung: löschen Sie keine Volumes von Images bzw. Container, die Sie bei anderen Modulen bzw. Lehrpersonen im Einsatz haben und weiterhin benötigen! Volumes beinhalten im Gegensatz zu Images und Container Daten, die Sie nicht mehr ohne Weiteres rekonstruieren können.**

2.2.3 Aufgabe 1 - Reinen Tisch machen

- Stoppen und entfernen Sie alle Container, die aktiv oder bereit sind.
- Entfernen Sie alle Docker-Images von Ihrem Computer.

2.3 Docker und Docker CLI-Befehle vertiefen

Achten Sie in den folgenden Aufgaben darauf, dass Sie möglichst nur offizielle Docker-Images Laden, die mit folgendem Label markiert sind.



2.3.1 Aufgabe 2 – Ein erstes Image laden

Gehen Sie auf Docker-Hub und suchen Sie nach dem Image «Hello World».

- Überfliegen Sie den Tab «Overview»
- Schalten Sie auf den Tab «Tags». Hier werden die verschiedenen Versionen angezeigt, wie auch der Befehl, um das Docker Image auf den eigenen Computer herunterzuladen.

```
$ docker pull hello-world:latest
```

```
latest: Pulling from library/hello-world
7050e35b49f5: Pull complete
Digest:
sha256:aa0cc8055b82dc2509bed2e19b275c8f463506616377219d9642221ab53cf9fe
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
```

Mit welchem Befehl können Sie überprüfen ob der Download erfolgreich war?

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	46331d942d63	10 months ago	9.14kB

Starten Sie nun einen Container basierend auf dem Image:

```
$ docker run 46331d942d63
```

Nun erscheint die folgende Nachricht:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

- The Docker client contacted the Docker daemon.
- The Docker daemon pulled the "hello-world" image from the Docker Hub.
- The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
- The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

...

Überprüfen Sie den Container und Sie werden sehen, dass der Container nicht mehr läuft. Er wurde zwar gestartet aber gleich wieder beendet. Dies ist offenbar ein Image bzw. ein Container der läuft um eine bestimmte Aufgabe zu erledigen und gleich wieder beendet wird.

```
$ docker ps
```

Überprüfen Sie den Container mit dem folgenden Befehl. Die Option `-a` zeigt alle Container, auch diejenigen die gestoppt wurden.

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b43b6bb7537a	46	"/hello"	6 seconds ago	Exited (0) 6 seconds ago		awesome_goldstine

Starten Sie den Container erneut mit folgendem Befehl:

```
$ docker start b43b6bb7537a
```

2.3.2 Aufgabe 3 – Nginx verwenden

Nginx ist ein Open-Source-Webserver der als Webserver oder auch als Reverse-Proxy, HTTP-Cache und Load-Balancer verwendet wird.

Einige namhafte Unternehmen setzen Nginx ein. Das entsprechende Docker-Image steht im Docker-Hub zur Verfügung und kann heruntergeladen und gestartet werden.

```
$ docker pull nginx:latest  
$ docker run nginx
```

2.3.3 Aufgabe 4 – Mit Docker-Compose Dateien umgehen

Docker Compose ist ein Werkzeug von Docker, das es Entwicklern ermöglicht, mehrere Docker-Container als eine einzelne Anwendung zu verwalten. Mit Docker Compose können Entwickler:innen eine einfache YAML-Datei erstellen, welche die Container, die für eine Anwendung erforderlich sind, mit den Abhängigkeiten und Konfigurationsoptionen beschreibt.

Die Datei "docker-compose.yml" ist die Konfigurationsdatei für Docker Compose, welche die Informationen zu den Containern, Netzwerken und Volumes beinhaltet und für eine Anwendung erforderlich sind.

Sie enthält Details darüber

- welche Image verwendet werden, welche Umgebungsvariablen für die Container gesetzt werden,
- welche Ports weitergeleitet werden,
- welche Volumes verwendet werden
- und welche Abhängigkeiten zwischen den Containern bestehen.

Mit Docker Compose können Sie eine gesamte Umgebung mit einem einzigen Befehl starten, anhalten oder entfernen. Das erleichtert das Erstellen, Testen und Bereitstellen von Anwendungen, die auf mehreren Containern bestehen.

- a) Schauen Sie sich die folgende docker-compose.yml Datei an und speichern Sie diese in einem Ordner ab, um mit ihr zu experimentieren.

```
version: '3'

services:
  mariadb:
    image: mariadb:10.8.6
    ports:
      - "3306:3306"
    volumes:
      - ./db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: 123456
      MARIADB_DATABASE: playerdb
      MARIADB_USER: playerdbuser
      MARIADB_PASSWORD: 123456

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    depends_on:
      - mariadb
    ports:
      - "8090:80"
    environment:
      PMA_HOST: mariadb
```

- b) Welche beiden Images wurden in dieser Komposition verwendet (Name und Version)?

-
-

- c) Starten Sie die Komposition mit folgendem Befehl und überprüfen Sie, ob die beiden Images laufen.

```
$ docker compose up
```

```
$ docker ps
```

Welche Konfiguration belegen die beiden Images?

Füllen Sie die folgende Tabelle aus.

Image Name	Id	Port-Host	Port-Container	Bemerkungen

Fahren Sie die Komposition herunter und löschen die Container sowie die benötigten Images und Volumes.

```
$ docker compose down
```

```
$ ...
```


3 Weiterführende Aufgaben

3.1 Aufgabe 5 – Nginx parametrisieren und starten

Sie können mit einem Nginx-Image einem Docker-Container erstellen, indem Sie das Image aus dem Docker-Hub verwenden und starten.

- a) Suchen Sie im Docker-Hub nach dem offiziellen Nginx-Image. Welches ist die letzte Version?
- b) Laden Sie das entsprechende Image herunter

```
$ docker pull nginx:latest
```

Starten Sie eine Instanz bzw. einen Container vom gerade heruntergeladenen Nginx-Image.

Der folgende Befehl verwendet die standardmässige Nginx-Konfiguration:

```
$ docker run --name mynginx -p 8070:80 -d nginx
```

--name mynginx

ist der Name des erstellten Containers basierend auf dem Nginx -Image

-p 8070:80

Diese Option weist Docker an, die vom Nginx-Image (Port 80) im Container bereitgestellten Ports dem angegebenen Port auf dem Docker-Host (Port 8070) zuzuordnen. Der erste Parameter gibt den Port im Docker-Host an, der zweite Parameter wird dem im Container exponierten Port zugeordnet

-d

Diese Option gibt an, dass der Container im getrennten Modus ausgeführt wird: Der Container wird weiter ausgeführt, bis er angehalten wird. Die Befehlszeile steht aber wieder zur Verfügung.

Überprüfen Sie den Container mit folgendem Befehl:

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ea920fa6d221	nginx	"/docker-entrypoint..."	4 seconds ago	Up 4 seconds	0.0.0.0:8070->80/tcp	mynginx

Öffnen Sie im Browser die URL <http://localhost:8070>

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

4 Lernkontrolle

4.1 Repetition 1 – Docker CLI-Befehle

Einen grossen Teil der folgenden Docker CLI-Befehle haben wir in diesen Dokument kennengelernt und repetiert.

→ Verbinden Sie die linke Spalte mit den Befehlen (a-o) mit den Auswirkungen der Befehle in der linken Spalte, indem Sie den richtigen Buchstaben einsetzen.

a	docker run		d	Entfernt einen gestoppten Container.
b	docker start		j	Zeigt die Ausgabe von stdout und stderr für einen Container an.
c	docker stop		c	Stoppt einen laufenden Container.
d	docker rm		n	Löscht ein Image vom Host.
e	docker ps		l	Startet alle Container einer Compose-Datei.
f	docker images		g	Lädt ein Image aus einem Docker-Repository herunter.
g	docker pull		f	Zeigt alle lokal vorhandenen Images an.
h	docker push		a	Startet einen neuen Container auf Basis eines Images.
i	docker exec		h	Lädt ein Image in ein Docker-Repository hoch.
j	docker logs		o	Stoppt ein Container.
k	docker build		b	Startet einen gestoppten Container.
l	docker compose up		e	Zeigt alle laufenden Container an.
m	docker compose down		k	Erstellt ein neues Image auf Basis eines Dockerfiles.
n	docker image rm		i	Führt einen Befehl in einem laufenden Container aus.
o	docker kill		m	Stoppt und entfernt alle Container einer Compose-Datei.

4.2 Repetition 2 – CLI-Befehle gruppieren

Gruppieren Sie die oben aufgelisteten Befehle nach der Anwendung auf Images, auf Container oder auf Kompositionen:

Docker Images Befehle	images, pull, push, build, image rm
Docker Container Befehle	run, start, stop, rm, ps, exec, logs
Docker Compose Befehle	compose up, compose down, kill

4.3 Repetition 3 - Repetitionsfragen Docker-CLI-Befehlen

1. Welcher Befehl wird verwendet, um einen neuen Container auf Basis eines Images zu starten?

- a) docker run
 - b) docker start
 - c) docker stop
 - d) docker rm
- a

2. Welcher Befehl wird verwendet, um einen gestoppten Container zu entfernen?

- a) docker run
 - b) docker start
 - c) docker stop
 - d) docker rm
- d

3. Welcher Befehl wird verwendet, um alle laufenden Container anzuzeigen?

- ☐ a) docker run
 - ☐ b) docker start
 - ☐ c) docker stop
 - ☐ d) docker ps
- d

4. Welcher Befehl wird verwendet, um ein Image in ein Docker-Repository hochzuladen?

- ☐ a) docker run
 - ☐ b) docker start
 - ☐ c) docker pull
 - ☐ d) docker push
- d

5. Welcher Befehl wird verwendet, um alle Container und Netzwerke zu stoppen und zu entfernen, die mit docker-compose up gestartet wurden?

- ☐ a) docker run
- ☐ b) docker start d
- ☐ c) docker stop
- ☐ d) docker-compose down

6. Welcher Befehl wird verwendet, um die Version von Docker auf dem Computer anzuzeigen?

- ☐ a) docker version
- ☐ b) docker info
- ☐ c) docker --version c
- ☐ d) docker status

7. Welcher Befehl wird verwendet, um eine Liste aller Images auf dem Computer anzuzeigen?

- ☐ a) docker images
- ☐ b) docker list a
- ☐ c) docker show
- ☐ d) docker display

8. Welcher Befehl wird verwendet, um den Standard-Docker-Registry zu ändern?

- ☐ a) docker registry
- ☐ b) docker change
- ☐ c) docker config d
- ☐ d) docker login

9. Welcher Befehl wird verwendet, um eine bestimmte Anzahl Log-Einträge eines Containers anzuzeigen?

- ☐ a) docker logs
- ☐ b) docker show logs a
- ☐ c) docker container logs
- ☐ d) docker inspect logs

10. Welcher Befehl wird verwendet, um den Zustand von einem bestimmten Container anzuzeigen?

- ☐ a) docker inspect
- ☐ b) docker status
- ☐ c) docker show a
- ☐ d) docker info